KAVACH

Duplicate Detection Agent

Deep Technical Flowchart & Step-by-Step Explanation

Purpose	Identify whether a newly submitted complaint plausibly refers to an existing incident, using multiple evidence signals.
Selected signals	Text meaning, location, category, time, and optional perceptual image hash.
Decision approach	Hybrid policy: DUPLICATE, REVIEW_REQUIRED, or NEW.
Prototype setup	Python + FastAPI + Pydantic + in-memory synthetic incident repository.

Design scope

This document describes only the Duplicate Detection Agent. It does not design the responsibilities of other KAVACH agents. The duplicate detector proposes matches and returns evidence; it does not delete complaints or perform destructive merges.

Version 1 • Week 1 prototype design

1. Master technical flowchart

Read from top to bottom. The agent validates the request, retrieves existing candidates, computes independent evidence signals, applies an evidence-aware scoring policy, and returns a reasoned decision.



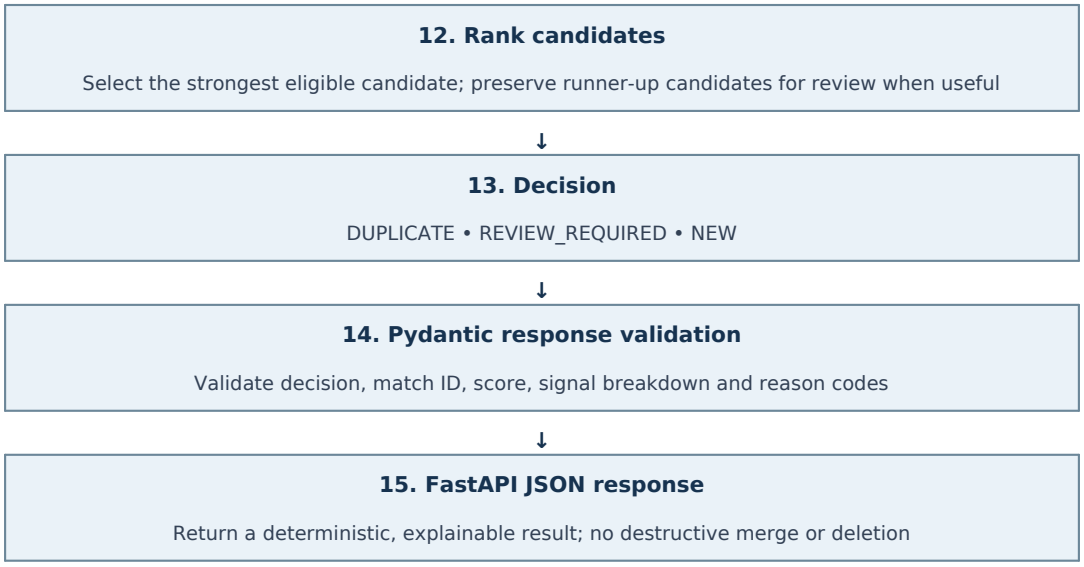


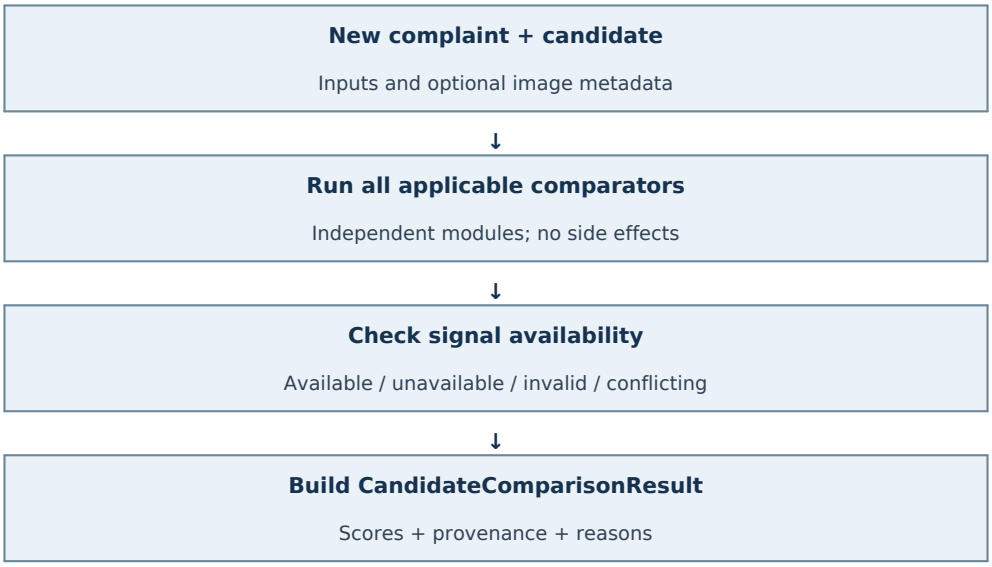
Figure 1. Master execution path. Branch-specific errors and optional image processing are expanded in the next sections.

2. Signal-comparison sub-flow

Each candidate is compared independently against the new complaint. Each comparator returns a score or an explicit unavailable/invalid status; missing evidence is never silently converted into a zero score.

Comparator	Technical steps	Output
TextComparator	Normalize text → compute semantic embeddings (or temporary stub) → cosine similarity → validate numeric range.	text_score; status; reason
LocationComparator	Compare known place/zone IDs; otherwise validate coordinates → calculate geographic distance → map to configured score.	location_score; distance/method; status
CategoryComparator	Compare category IDs → consult explicit compatibility map → distinguish same, compatible, incompatible and missing.	category_score; compatibility status
TimeComparator	Parse timestamps with timezone rules → calculate absolute time gap → apply configurable decay/window.	time_score; delta; status
ImageHashComparator	If both valid images/hashes exist → compute perceptual hashes consistently → Hamming distance → normalized similarity.	image_hash_score; distance; status

Technical branch flow



3. Scoring and hybrid decision flow

The numerical score is a ranking signal, not a probability. Initial weights and thresholds are hypotheses to validate against labelled examples.

Illustrative core score

```
S_core = 0.40 × S_text
       + 0.30 × S_location
       + 0.15 × S_category
       + 0.15 × S_time
```

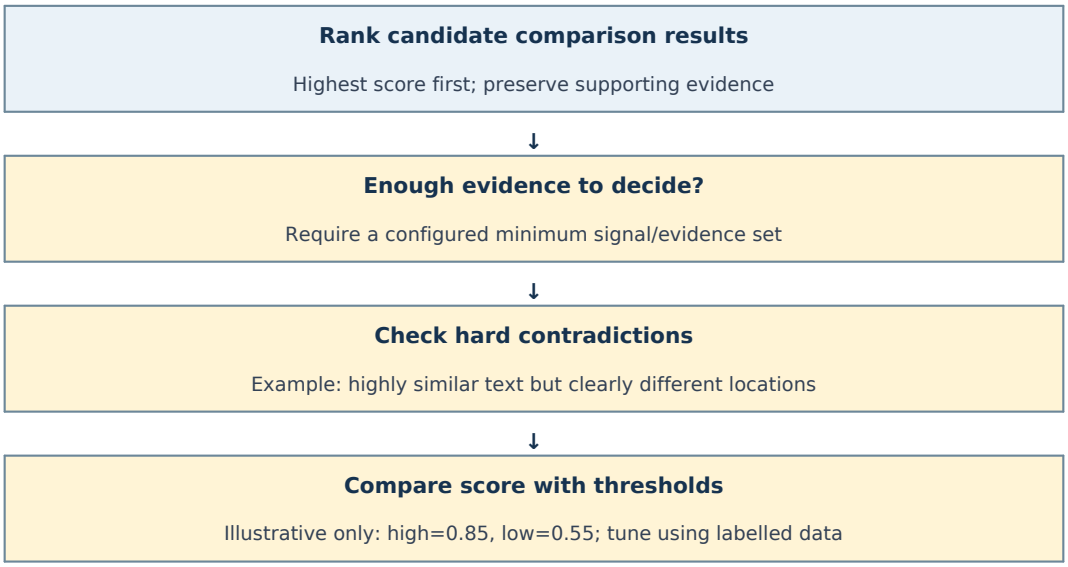
For a candidate with scores text=0.91, location=1.00, category=1.00 and time=0.95, the illustrative combined score is 0.949. This does not mean a 94.9% probability of duplication.

Available-signal normalization

```
S = sum(weight_i × score_i for available signals)
  / sum(weight_i for available signals)
```

Use this only with an evidence sufficiency policy. For example, one available signal alone must not be enough to produce an automatic duplicate decision. Missing signals remain missing; they are not scored as zero.

Decision sub-flow



DUPLICATE	REVIEW_REQUIRED	NEW
High score + enough evidence + no disqualifying conflict. Return proposed match ID.	Borderline score, missing evidence or conflicting signals. Return candidates and reasons.	No credible match among searched candidates. Do not claim global certainty.

4. Step-by-step explanation of the flowchart

Step 1 — Structured complaint arrives

The agent should receive structured fields rather than trying to interpret arbitrary incoming UI data itself. A useful contract includes complaint_id, description, category, available location data, reported_at and optional image reference/hash. Original values should be retained for audit and debugging.

Step 2 — FastAPI endpoint

A dedicated route such as POST /api/v1/duplicate-detection/check accepts the request and delegates to a service. The route should not contain all scoring logic. This keeps HTTP concerns separate from comparison policy.

Step 3 — Pydantic validation

Validate field types, required identifiers, timestamp parsing, category representation, coordinate ranges and optional image-hash format. Invalid requests should fail clearly rather than silently entering the scoring pipeline. Semantic validity checks belong after basic schema validation.

Step 4 — DuplicateDetectionService

This service controls the order of operations and gathers results. It should call deterministic functions and repository interfaces; it should not hide all logic in one large function or require a live API call for every comparator.

Step 5 — Normalization and context

Normalize text consistently, standardize category IDs, parse timestamps with timezone awareness and normalize location identifiers. Preserve missing-field flags and original inputs. Normalization must not invent missing location or time data.

Step 6 — Candidate retrieval

Search the prototype repository for existing incidents. The initial implementation can scan a list of synthetic records. Exclude the same complaint ID, records that should not be compared under the current policy, and malformed records. Candidate filtering should be conservative enough not to remove plausible matches.

Step 7 — Empty candidate set

If no candidates are returned, the agent may respond NEW with NO_CANDIDATES_FOUND. This means no match was found in the dataset and scope searched; it is not proof that no duplicate exists elsewhere.

Step 8 — Independent comparisons

For every candidate, calculate text, location, category and time evidence, plus image-hash evidence if both sides have usable images. Comparators should return structured results, not just bare floats, so the scoring policy knows whether a signal is unavailable, invalid or contradictory.

Step 9 — CandidateComparisonResult

Store the candidate ID, individual scores, availability statuses, details such as distance/time gap/hash distance, and reason codes. This is the central intermediate structure between comparison and decision logic.

Step 10 — ScoringPolicy

Combine the available core signals using configurable weights. If a signal is missing, normalize over available weights but enforce minimum evidence requirements. A candidate with only text similarity

should not automatically be declared a duplicate.

Step 11 — HybridDecisionPolicy

Apply thresholds together with evidence rules. Strong contradictions can force REVIEW_REQUIRED even when the aggregate score is high. Image hash may support a match, but a similar hash should not independently prove duplicate identity.

Step 12 — Candidate ranking

Rank candidates by their score and evidence. If multiple candidates score similarly, the system should preserve ambiguity and return more than one candidate for review rather than selecting a false certainty.

Step 13 — Decision outcomes

DUPLICATE means a sufficiently strong match was identified; REVIEW_REQUIRED means evidence is ambiguous or conflicting; NEW means no credible candidate met the duplicate policy. Each result should have explicit reason codes.

Step 14 — Response validation

Validate the final response schema: decision enum, matched incident ID when appropriate, numeric score range, per-signal breakdown and reasons. Do not return NaN/infinite values or unstructured model-generated prose.

Step 15 — JSON response

Return the result to the caller. The agent proposes a match and explains it. It does not delete or overwrite complaints, perform a destructive merge, or claim that the physical incident has been verified.

5. Example request and response contracts

Illustrative input JSON


```
{
  "complaint_id": "INC-127",
  "description": "Garbage is spilling out of the bin at the entrance",
  "category": "WASTE_OVERFLOW",
  "location": {
    "facility_id": "SITE-01",
    "location_id": "MAIN_GATE",
    "latitude": 23.2599,
    "longitude": 77.4126
  },
  "reported_at": "2026-10-12T09:05:00+05:30",
  "image_hash": "a1b2c3d4"
}
```


The coordinates and image hash above are example values only. Production schema should distinguish absent, invalid and verified fields, and should not trust client-supplied metadata without validation.

Illustrative duplicate response


```
{
  "complaint_id": "INC-127",
  "decision": "DUPLICATE",
  "matched_incident_id": "INC-104",
  "score": 0.949,
  "signals": {
    "text": {"score": 0.91, "status": "AVAILABLE"},
    "location": {"score": 1.0, "status": "AVAILABLE"},
    "category": {"score": 1.0, "status": "AVAILABLE"},
    "time": {"score": 0.95, "status": "AVAILABLE"},
    "image_hash": {"score": null, "status": "UNAVAILABLE"}
  },
  "reasons": [
    "HIGH_TEXT_SIMILARITY",
    "SAME_KNOWN_LOCATION",
    "SAME_CATEGORY",
    "CLOSE_REPORT_TIME"
  ]
}
```


Keep score semantics explicit: 0.949 is a combined ranking score under this configured policy, not a calibrated 94.9% probability. In a real response, per-signal statuses and score availability must agree.

Suggested Pydantic model outline

```
class DuplicateDecision(str, Enum):
    DUPLICATE = "DUPLICATE"
    REVIEW_REQUIRED = "REVIEW_REQUIRED"
    NEW = "NEW"

class SignalResult(BaseModel):
    score: float | None
    status: str
    reason: str | None = None

class DuplicateDetectionResponse(BaseModel):
    complaint_id: str
    decision: DuplicateDecision
    matched_incident_id: str | None
    score: float | None
    signals: dict[str, SignalResult]
    reasons: list[str]
```

6. Suggested Python module structure

```
app/
  main.py
  api/
    duplicate_routes.py
  schemas/
    duplicate_request.py
    duplicate_response.py
  services/
    duplicate_detection_service.py
  retrieval/
    candidate_retriever.py
  comparators/
    text_comparator.py
    location_comparator.py
    category_comparator.py
    time_comparator.py
    image_hash_comparator.py
  policy/
    scoring_policy.py
    hybrid_decision_policy.py
  repositories/
    base.py
    in_memory.py
  config/
    duplicate_settings.py
tests/
  test_request_validation.py
  test_text_comparator.py
  test_location_comparator.py
  test_category_comparator.py
  test_time_comparator.py
  test_image_hash_comparator.py
  test_scoring_policy.py
  test_hybrid_decision_policy.py
  test_duplicate_endpoint.py
```

Module responsibilities

Module	Owns	Must not own
API route	HTTP request/response and status codes	Scoring rules and data mutation
Service	Pipeline orchestration	Low-level details for every signal
Retriever/repository	Read candidate records	Deciding if records are duplicates
Comparators	One signal each; return structured evidence	Final duplicate decision
Scoring policy	Weighted aggregation and evidence checks	HTTP handling or deleting records
Decision policy	Outcome from score + rules	Rewriting original complaint

7. Error handling and edge cases

Invalid request

Return a clear validation error; do not run comparisons.

No candidates

Return NEW with NO_CANDIDATES_FOUND and make the search scope explicit.

Missing text

Mark text signal unavailable; do not compare empty strings as a strong match.

Missing location

Location signal unavailable; never assume distance equals zero.

Invalid timestamp

Time signal unavailable or request validation error, according to the contract.

One missing image

Image hash comparison unavailable; do not penalize the candidate.

Corrupt image

Record an image-processing failure reason; continue other comparisons if the request remains valid.

Comparator failure

Return an explicit signal error or controlled service error; never silently substitute a high score.

Tie between candidates

Return REVIEW_REQUIRED or multiple candidates if policy cannot safely distinguish them.

Empty dataset

Return NEW; distinguish an empty repository from a repository failure.

Duplicate input ID

Exclude self-comparison; if ID collision semantics are uncertain, reject or flag it explicitly.

8. Testing and acceptance criteria

Test the individual comparator functions first, then the scoring/decision policy, and finally the FastAPI endpoint. Keep synthetic test records deterministic so results do not change unexpectedly between runs.

Test scenario	Expected behavior
Paraphrased descriptions, same location/category/time	Strong candidate match when sufficient evidence exists
Similar text but clearly different locations	Do not automatically mark duplicate; apply contradiction/review rule
Same place, incompatible categories	Review or non-duplicate based on explicit compatibility policy
Same image hash, other evidence differs	Image match is supporting evidence, not sole proof
Image absent or corrupt	Other signals continue; image is unavailable, not score zero
No candidates in repository	NEW with explicit reason code
Only one signal available	Evidence sufficiency rule prevents unjustified automatic duplicate
Two equally strong candidates	Review or return ambiguity; don't invent certainty
Malformed coordinates/time	Reject or mark that signal invalid according to schema policy
Response contains NaN or score outside range	Response validation fails; no invalid JSON result

Metrics to evaluate

• Precision: Of the cases classified DUPLICATE, how many are truly duplicates?

• Recall: Of the labelled true duplicates, how many did the agent find?

• False-positive rate: How often does it incorrectly link distinct incidents?

• Review rate: How often is the agent uncertain and requests review?

Choose thresholds based on labelled examples and the cost of false matches. Initial weights and thresholds are configuration hypotheses, not production truth.

9. Technical trade-offs and safeguards

Prototype scan vs database/vector search

A full in-memory scan is simple and transparent for a small synthetic dataset. It does not scale indefinitely and disappears at process restart. Introduce a persistent database/search index only when the prototype's matching logic is validated.

Embedding model vs keyword similarity

Embeddings handle paraphrases better but add a model dependency, initial download/storage and runtime considerations. Keep the text comparator interface replaceable. Do not require embeddings to test the overall pipeline.

Perceptual hash vs image embeddings

Perceptual hashes are lightweight and can detect near-identical image files, but they do not establish that two different photos show the same real-world event. Your current design intentionally chooses

hashes.

Weighted score vs calibrated probability

A weighted score is interpretable and easy to tune but is not a probability. Calibration would require a labelled dataset and a separate evaluation process.

Automatic decision vs review

Automation reduces repetitive work; review limits harm from ambiguous or contradictory evidence. Preserve candidate reasons so the outcome is explainable.

Read-only matching vs mutation

The detector should propose a match and return IDs. Destructive merging or deletion is outside this agent's boundary.

10. Definition of done for the prototype

- FastAPI endpoint accepts a validated request and returns a validated response.
- Candidate retrieval works against a synthetic in-memory repository.
- Each comparison signal is a separate, testable module.
- Unavailable signals are represented explicitly and do not become false zero scores.
- Scoring weights and thresholds are configurable.
- The hybrid policy supports DUPLICATE, REVIEW_REQUIRED and NEW.
- Every decision includes reason codes and a signal breakdown.
- Unit tests cover positive, negative, missing-data, conflict and tie scenarios.
- No original complaint is deleted or overwritten by this agent.
- The whole prototype runs without a frontend or persistent database.

Final design summary

FastAPI receives and validates a structured complaint; a service retrieves existing candidates from memory; independent comparators produce evidence; a scoring policy combines available core signals; a hybrid policy applies thresholds, evidence sufficiency and contradiction rules; Pydantic validates the explainable result. The agent proposes a match but does not delete or merge complaint records.